

YIĞINLAR

- Java / Spring Boot
- C# / .NET
- Ortak mimari

İÇİNDEKİLER

Giriş

- 01 · Çalışma zamanı: JIT & GC
- 02 · Ahead-of-time derleme
- 03 · Bağlantı havuzlama
- 04 · N+1 sorgu problemi
- 05 · Çok katmanlı önbellekleme
- 06 · Asenkron / reaktif mimari
- 07 · Mesaj kuyruğuyla ayrıştırma
- 08 · CQRS: okuma-yazma ayrımı
- 09 · Devre kesici & dayanıklılık
- 10 · Yatay ölçekleme
- 11 · Profilleme araçları

GENEL REFERANS · PROJE-BAĞIMSIZ

Kurumsal Backend Optimizasyonu: Java/Spring Boot & C#/.NET

Bu doküman PsGameTranslator'dan tamamen bağımsız — Java/Spring Boot ve C#/.NET dünyasında yaygın backend performans sorunlarını ve mimari çözümlerini yan yana anlatıyor. Amaç: bir teknikle bir yığında karşılaşınca diğesinde de tanıyabilmen. Her bölüm önce *neden* işe yaradığını (mimari), sonra *her iki yığında nasıl uygulandığını* (kod) gösteriyor.

- Java / Spring Boot örnekleri
- C# / .NET (Core) örnekleri
- Yığından bağımsız mimari kavram

Çalışma zamanı temelleri: JIT ve Garbage Collector

Hem JVM hem CLR yönetilen (managed) çalışma zamanlarıdır — ikisi de kodu anında (JIT) derler ve belleği otomatik toplar. Farklı olan, bu iki sürecin nasıl ayarlanabildiği.

JIT: sıcak yol (hot path) optimizasyonu

Kod ilk çalıştığında yorumlanır/az optimize derlenir. Sık çalışan metodlar ("sıcak") zamanla daha agresif optimize edilerek yeniden derlenir — bu yüzden bir servisin ilk birkaç isteği (soğuk başlangıç) her zaman sonraki isteklerden yavaştır.

JVM — G1GC / ZGC

Modern JVM'lerde varsayılan **G1GC**, büyük heap'lerde milisaniye altı duraklama hedefler. Çok büyük heap (>32GB) veya duraklama-kritik servislerde **ZGC** / **Shenandoah** tercih edilir — duraklama süresi heap boyutundan bağımsızdır.

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-Xms2g -Xmx2g          # min=max: heap resize duraklamasını önler
-XX:+UseZGC             # çok büyük heap / ultra-düşük duraklama gerekirse
```

◆ CLR — WORKSTATION VS SERVER GC

Server GC çekirdek başına bir heap açar, throughput'u artırır ama bellek izini büyütür — sunucu/API iş yükü için uygun. Workstation GC (özellikle concurrent modda) masaüstü/düşük eşzamanlı servislerde daha az bellek, daha öngörülebilir duraklama verir.

```
// .csproj veya runtimeconfig.json
<ServerGarbageCollection>true</ServerGarbageCollection>
<ConcurrentGarbageCollection>true</ConcurrentGarbageCollection>
// Yüksek eşzamanlı ASP.NET Core API → Server GC true
// Masaüstü/az eşzamanlı araç → Server GC false (varsayılan)
```

Ortak ilke: Her iki runtime'da da GC ayarlarının altın kuralı aynı — önce ne kadar ve ne sıklıkla nesne ayırdığını ölç (allocation rate), sonra GC'yi buna göre ayarla. GC ayarını kör kör değiştirmek genelde işe yaramaz; asıl kazanç genelde gereksiz ayırmayı *azaltmaktan* gelir (obje havuzları, struct kullanımı, streaming API'ler).

Ahead-of-time (AOT) derleme

JIT'in "ilk isteği yavaş" sorununu tamamen ortadan kaldıran yaklaşım: kodu dağıtımdan önce, çalışma zamanında değil, derleme sırasında makine koduna çevirmek.

GRAALVM NATIVE IMAGE

Spring Boot uygulamasını gerçek bir native binary'e derler — JVM başlatma maliyeti ve JIT ısınma süresi sıfırlanır. Başlangıç süresi saniyelerden milisaniyelere iner; container/serverless ortamlarda (soğuk başlangıç kritikse) büyük fark yaratır.

```
<plugin>
  <groupId>org.graalvm.buildtools</groupId>
  <artifactId>native-maven-plugin</artifactId>
</plugin>
# mvn -Pnative native:compile
# Sonuç: ~50ms başlangıç (vs ~2-3sn JVM ısınması)
```

.NET NATIVE AOT

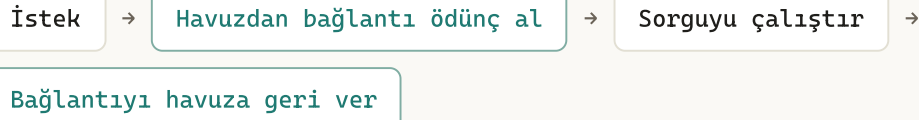
.NET 8+'da `PublishAot` ile aynı fikir — JIT'siz, doğrudan çalıştırılabilir native binary. Daha küçük dağıtım boyutu, anında başlangıç, daha az bellek izi (JIT altyapısının kendisi de bellek kaplamıyor).

```
<PropertyGroup>
  <PublishAot>true</PublishAot>
</PropertyGroup>
<!-- dotnet publish -c Release -r win-x64 -->
```

Takas: AOT, reflection-yoğun kütüphanelerle (bazı ORM'ler, DI container'lar, serileştiriciler) sorun çıkarabilir çünkü "hangi kodun çalışacağı" derleme zamanında bilinmelidir. Spring Boot 3+ ve ASP.NET Core 8+ bunu büyük ölçüde çözdü ama üçüncü parti kütüphane uyumluluğunu migrasyondan önce kontrol etmek gerekir.

Bağlantı havuzlama (connection pooling)

Her istekte yeni bir veritabanı bağlantısı açmak (TCP handshake + auth) tek başına isteğin geri kalanından daha pahalı olabilir.



Bağlantı kapatılmaz, sadece havuza iade edilir — bir sonraki istek onu yeniden kurmadan kullanır.

● HIKARICP (SPRING BOOT VARSAYILANI)

```
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.connection-timeout=3000
spring.datasource.hikari.idle-timeout=600000
```

`maximum-pool-size` 'ı çekirdek sayısının katı gibi büyütmek genelde yanlıştır — HikariCP'nin kendi dokümantasyonu bile "az bağlantı, çok istek" formülünü önerir çünkü veritabanı sunucusu zaten sınırlı eşzamanlılıkla çalışır.

◆ ADO.NET / EF CORE (DAHİLİ HAVUZLAMA)

```
"ConnectionStrings": {
  "Default": "Server=...;Max Pool Size=50;Min Pool Size=5;
             Connection Timeout=15;"
}
// EF Core: DbContext'i her istek için Scoped olarak kaydet,
// bağlantı havuzu connection string düzeyinde otomatik yönetilir.
services.AddDbContext<AppDbContext>(o => o.UseSqlServer(connStr),
  ServiceLifetime.Scoped);
```

Bölüm 04

N+1 sorgu problemi

ORM'lerin en yaygın performans tuzağı — 1 liste sorgusu, ardından her öge için ayrı ayrı N sorgu daha.

Neden olur?

`Order` listesi çekip her `order.getCustomer()` 'a eriştiğinde, ORM lazy-loading ile o an ayrı bir sorgu atar. 100 sipariş = 1 + 100 = 101 sorgu. Çözüm: ilişkiyi baştan, tek sorguda (join ile) getirmek.

JPA / HIBERNATE

```
// Kötü: N+1 tetikler
List<Order> orders = orderRepository.findAll();
orders.forEach(o → o.getCustomer().getName()); // her biri ayrı sorgu

// İyi: JOIN FETCH ile tek sorguda getir
@Query("SELECT o FROM Order o JOIN FETCH o.customer")
List<Order> findAllWithCustomer();

// Ya da @EntityGraph
@EntityGraph(attributePaths = {"customer"})
List<Order> findAll();
```

◆ ENTITY FRAMEWORK CORE

```
// Kötü: N+1 tetikler (lazy loading açıksa)
var orders = context.Orders.ToList();
orders.ForEach(o ⇒ Console.WriteLine(o.Customer.Name));

// İyi: Include ile eager loading, tek sorguda JOIN
var orders = context.Orders
    .Include(o ⇒ o.Customer)
    .AsNoTracking() // salt-okunur sorguda change-tracking kapat
    .ToList();
```

Ek ipucu (her iki yığın): Salt-okunur (read-only) sorgularda change-tracking'i kapatmak (`AsNoTracking()` / Hibernate'de `@Transactional(readOnly = true)`) ORM'nin "bu nesneyi değişiklik için izlememe" yükünü ortadan kaldırır — genelde gözden kaçan ama ucuz bir kazanç.

Tek bir önbellek yetmez — istek sürecinin farklı noktalarında farklı ömürde önbellek katmanları olur.



L1 en hızlı ama süreç yeniden başlayınca/yatay ölçeklenince kaybolur ve tutarsızlaşabilir; L2 süreçler arası paylaşılır ve tutarlı kalır ama ağ gecikmesi taşır.

● SPRING CACHE + CAFFEINE + REDIS

```
@Cacheable(value = "products", key = "#id")
public Product getProduct(Long id) { ... }

// application.yml
spring.cache.type: caffeine
spring.cache.caffeine.spec: maximumSize=5000,expireAfterWrite=10m

// İkinci katman için spring-boot-starter-data-redis +
// @Cacheable(cacheManager = "redisCacheManager")
```

◆ IMEMORYCACHE + IDISTRIBUTEDCACHE (REDIS)

```
public async Task<Product> GetProductAsync(int id)
{
    if (_memoryCache.TryGetValue(id, out Product p)) return p;

    var cached = await _distributedCache.GetStringAsync($"product:{id}");
    if (cached is not null)
    {
        p = JsonSerializer.Deserialize<Product>(cached);
        _memoryCache.Set(id, p, TimeSpan.FromMinutes(2));
        return p;
    }

    p = await _db.Products.FindAsync(id);
    await _distributedCache.SetStringAsync($"product:{id}",
        JsonSerializer.Serialize(p), new() { SlidingExpiration = TimeSpan.FromMinutes(10) });
    _memoryCache.Set(id, p, TimeSpan.FromMinutes(2));
    return p;
}
```

Asenkron / reaktif mimari

Bir thread'i G/Ç beklerken bloke etmemek — az sayıda thread'le çok sayıda eşzamanlı isteği karşılamanın yolu.

Neden thread bloklamak pahalı?

Her thread işletim sisteminde ~1MB stack ve context-switch maliyeti taşır. Senkron bir servis 10.000 eşzamanlı isteği karşılamak için 10.000 thread ister (pratikte imkansız). Asenkron model, bir thread G/Ç beklerken (DB sorgusu, HTTP çağrısı) o thread'i havuza iade eder — sonuç gelince uygun bir thread devam ettirir. Az thread, çok eşzamanlı iş.

🍷 SPRING WEBFLUX (REACTOR)

```
@GetMapping("/products/{id}")
public Mono<Product> getProduct(@PathVariable Long id) {
    return productRepository.findById(id)           // non-blocking DB
        .flatMap(p -> enrichWithPricing(p))         // başka bir async çağrı
        .timeout(Duration.ofSeconds(3));
}
// Netty event-loop üzerinde çalışır – sabit sayıda thread,
// binlerce eşzamanlı bağlantı.
```

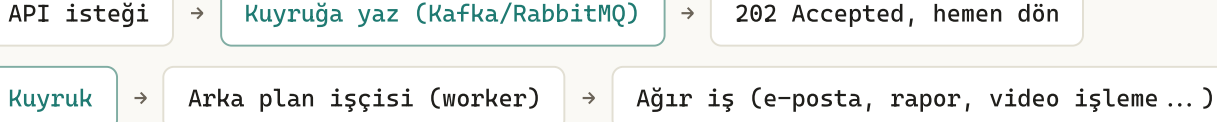
◆ ASYNC/AWAIT (ASP.NET CORE)

```
[HttpGet("products/{id}")]
public async Task<Product> GetProduct(int id)
{
    var product = await _db.Products.FindAsync(id); // thread pool'a iade edilir
    var pricing = await _pricingClient.GetAsync(id); // beklerken de öyle
    return Merge(product, pricing);
}
// Kestrel de aynı prensiple çalışır: I/O completion port'lar
// üzerinden az thread'le çok bağlantı.
```

Yaygın hata (her iki yığın): Asenkron bir API'yi senkron gibi çağırmak (`.block()` Reactor'da, `.Result/.Wait()` .NET'te) tüm kazanımı sıfırlar ve deadlock riski yaratır. Asenkron zincir baştan sona asenkron kalmalı.

Mesaj kuyruğuyla ayırıştırma (decoupling)

Yığından bağımsız bir mimari desen — bir isteğin cevabını, o isteğin tetiklediği ağır işin bitmesine bağlamamak.



Kullanıcı ağır işin bitmesini beklemez; sistem yük altında kuyruk büyür ama çöküp istek kaybetmez.

Ne zaman kullanılır?

İş, kullanıcının o an cevap almasını gerektirmeyen bir yan etkiyse (bildirim gönderme, rapor üretme, resim/video işleme, üçüncü parti API senkronizasyonu) senkron tutmak isteği gereksiz yere yavaşlatır ve üçüncü parti servisin yavaşlığını/çökmesini doğrudan kullanıcıya yansıtır. Kuyruk bu bağımlılığı zamanla ayırır (temporal decoupling).

KAVRAM	JAVA/SPRING	.NET
Kuyruk/broker	Kafka, RabbitMQ (spring-kafka, spring-amqp)	Azure Service Bus, RabbitMQ (MassTransit), Kafka
Arka plan işçisi	@KafkaListener / @RabbitListener	BackgroundService / IHostedService
Yeniden deneme	Spring Retry / broker seviyesi DLQ	Polly retry policy / broker seviyesi DLQ

CQRS: okuma ve yazma yollarını ayırma

Aynı veri modeli hem okuma hem yazma için optimize edilemez — CQRS bu ikisini mimari olarak ayırır.

Temel fikir

Command (yazma) tarafı tutarlılığa, doğrulamaya ve normalize edilmiş veri modeline önem verir. Query (okuma) tarafı hıza ve genelde denormalize, sorgu-dostu bir şekle önem verir. İkisini aynı entity/repository ile

yapmaya çalışmak her iki tarafı da orta seviyede tutar — ayırmak ikisini de kendi amacına göre optimize etmeni sağlar.

Yazma isteği → Command handler → normalize edilmiş DB → Olay yayınla

Olay → Okuma modelini güncelle (denormalize/materialized view) →

Sorgular buradan, hızlı okur

Ne zaman gerekli değil: Basit CRUD uygulamalarında CQRS gereksiz karmaşıklıktır. Okuma yükü yazma yükünden çok daha büyükse (tipik: 100:1 okuma/yazma oranı) ve okuma tarafı farklı şekillerde sorgulanıyorsa (raporlama, arama, dashboard) mantıklı hale gelir.

Bölüm 09

Devre kesici (circuit breaker) ve dayanıklılık

Yavaş/çökmüş bir bağımlılığın tüm sistemi aşağı çekmesini önlemek — bu bir performans deseni değil ama performans çöküşünü önler.

Kademeli çöküş (cascading failure)

Bir alt servis yavaşladığında, ona bağımlı her istek de yavaşlar, thread'ler o yavaş çağrıda birikir, thread pool tükenir, sonunda bağımsız endpoint'ler bile cevap veremez hale gelir. Devre kesici, art arda başarısız/yavaş çağrılardan sonra "devreyi açar" — bir süre o servise hiç istek göndermez, hızlıca hata döner, sistemin geri kalanını korur.

RESILIENCE4J

```
@CircuitBreaker(name = "pricingService", fallbackMethod = "fallback")
public Pricing getPricing(Long productId) {
    return pricingClient.fetch(productId);
}
private Pricing fallback(Long productId, Throwable t) {
    return Pricing.cached(productId); // bayat ama hızlı cevap
}
```

◆ POLLY

```
var breaker = Policy
    .Handle<HttpRequestException>()
    .CircuitBreakerAsync(
        exceptionsAllowedBeforeBreaking: 5,
        durationOfBreak: TimeSpan.FromSeconds(30));

var pricing = await breaker.ExecuteAsync(() => _pricingClient.GetAsync(productId));
```

Bölüm 10

Yatay ölçekleme: durum tutmama (statelessness)

Bir sunucuyu büyütmek (dikey) yerine sunucu sayısını artırmak (yatay) — ama bu, uygulamanın durumsuz tasarlanmasını gerektirir.

Neden durum tutmamak şart?

Bir kullanıcının oturum verisi tek bir sunucunun belleğinde tutulursa (in-memory session), bir sonraki isteği farklı bir sunucuya (load balancer arkasında) düşünce o veri yok gibi görünür. Durumu paylaşılan bir yere (Redis, veritabanı) taşımak, herhangi bir sunucunun herhangi bir isteği karşılayabilmesini sağlar — bu da yatay ölçeklemenin ön koşuludur.

DURUM TÜRÜ	JAVA/SPRING ÇÖZÜMÜ	.NET ÇÖZÜMÜ
Oturum (session)	Spring Session + Redis	ASP.NET Core Distributed Session + Redis
Kimlik doğrulama durumu	Durumsuz JWT (sunucu bir şey saklamaz)	Durumsuz JWT (aynı prensip)
Kısa süreli önbellek	L1'i (Caffeine) kabul et, ama gerçeği her zaman L2/DB'den doğrula	Aynı prensip — IMemoryCache asla "tek gerçek kaynak" olmamalı

Bölüm 11

Profilleme araçları

Her iki dünyada da "önce ölç" prensibinin somut karşılığı.

İHTİYAÇ	JAVA/JVM	.NET/CLR
Canlı CPU/bellek profili	JFR (Java Flight Recorder) + JDK Mission Control, async-profiler	dotnet-trace, dotnet-counters, PerfView
Heap dump analizi	Eclipse MAT	dotnet-dump, WinDbg + SOS
APM / dağıtık izleme	Micrometer + Zipkin/Jaeger, Datadog APM	Application Insights, OpenTelemetry .NET
Yük testi	Gatling, JMeter	k6, NBomber

Ortak metodoloji: Hangi araç olursa olsun sıra hep aynı — üretim benzeri yükte profil çıkar, en çok zaman/bellek harcayan 3-5 noktayı bul, sadece onları optimize et, tekrar ölç. Mikro-benchmark'lara (JMH, BenchmarkDotNet) güvenmeden önce gerçek trafiği yansıtan bir profil çıkarmak, yanlış darboğazı optimize etmeni önler.

Bu doküman herhangi bir projeye bağlı değildir — Java/Spring Boot ve C#/.NET'te ortak backend performans sorunlarının mimari çözümlerini karşılaştırmalı anlatır. PsGameTranslator'a özel öneriler için "Kaynak Optimizasyon Rehberi", genel optimizasyon teorisi için "Performans Optimizasyonu: Kavramlar ve Yöntemler" belgesine bakın.